

Clasificación de imágenes con tres modelos basados en VGG16

Guillermo Tinoco Ramos

27 de julio de 2026

Resumen

Se comparan tres clasificadores de flores que conservan intacta y congelada la sección convolucional de VGG16. Los modelos difieren solamente en sus capas densas. La comparación utiliza Accuracy, Precision, Recall y F1 Score sobre una partición de prueba independiente.

1. Conjunto de datos

Se utiliza `tf_flowers:3.0.1` de TensorFlow Datasets, compuesto por 3,670 imágenes RGB de cinco clases: dandelion, daisy, tulips, sunflowers y roses [1]. Como el catálogo ofrece un único split, se genera una partición estratificada y determinista de 70 % entrenamiento, 15 % validación y 15 % prueba con semilla 42. Esta decisión conserva aproximadamente la proporción de cada clase y evita emplear prueba para seleccionar el modelo. Las imágenes tienen dimensiones y condiciones variables, por lo que se redimensionan a 224×224 y se aplica `preprocess_input` de VGG16.

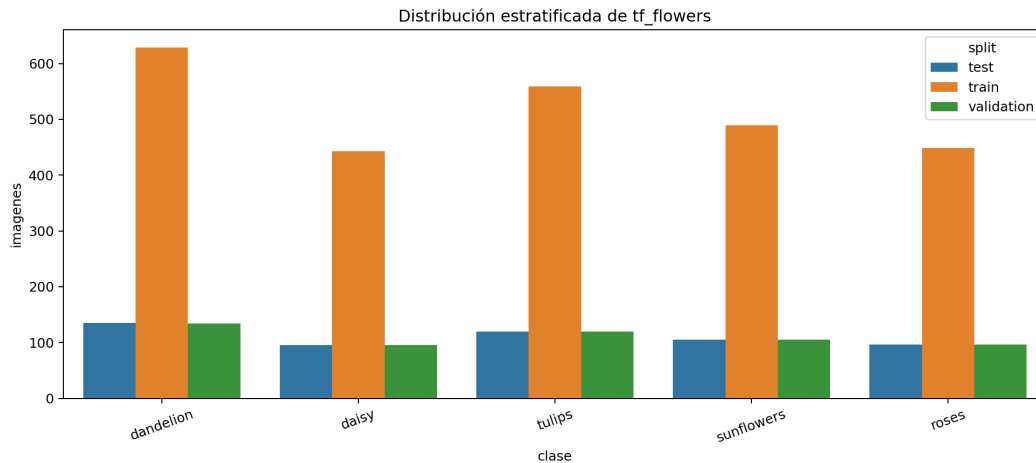


Figura 1: Distribución por clase y partición.

2. Metodología

Los tres modelos emplean VGG16 preentrenada en ImageNet con `include_top=False`. Sus 13 capas convolucionales permanecen congeladas; después se aplica `GlobalAveragePooling2D`. Todos usan Adam, tasa 10^{-3} , entropía cruzada categórica dispersa, lotes de 16, máximo 30 épocas, early stopping y el mismo conjunto de datos aumentado.

3. Modelos propuestos

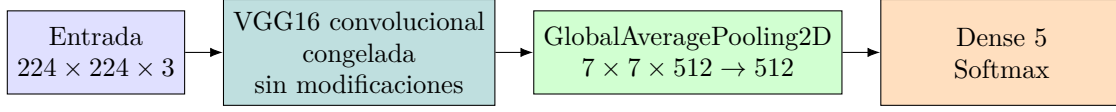


Figura 2: Arquitectura del Modelo A. Únicamente cambia el cabezal naranja.

El Modelo A es la línea base: sus cinco salidas aprenden una frontera lineal sobre las 512 características, minimizando parámetros y riesgo de sobreajuste.

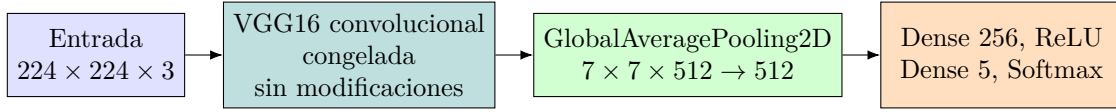


Figura 3: Arquitectura del Modelo B. Únicamente cambia el cabezal naranja.

El Modelo B agrega 256 neuronas ReLU para aprender combinaciones no lineales con una capacidad intermedia.

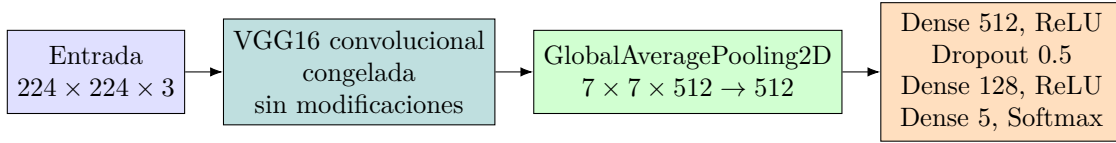


Figura 4: Arquitectura del Modelo C. Únicamente cambia el cabezal naranja.

El Modelo C incrementa la capacidad y usa Dropout de 0.5 para reducir coadaptación y controlar el sobreajuste en un dataset moderado.

4. Métricas

Para cada clase se interpreta el problema como uno-contra-el-resto:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Precision} = \frac{TP}{TP + FP}, \quad (1)$$

$$\text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = 2 \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (2)$$

Se presentan valores por clase y promedios macro y ponderado. F1 macro es el criterio principal porque otorga el mismo peso a las cinco clases. Las filas de la matriz de confusión son observaciones reales y las columnas predicciones.

5. Resultados

Cuadro 1: Resultados sobre el conjunto de prueba.

Modelo	Accuracy	Precision macro	Recall macro	F1 macro
A	0.8802	0.8798	0.8814	0.8799
B	0.8784	0.8785	0.8777	0.8775
C	0.8675	0.8713	0.8644	0.8652

Selección del mejor modelo. El Modelo A obtuvo el mayor F1 macro (0.8799), con Accuracy 0.8802. Por tanto, bajo la regla definida antes del experimento, es el modelo seleccionado. La interpretación por clase y los posibles indicios de sobreajuste deben contrastarse con las matrices de confusión y las curvas presentadas.

Análisis por clase. En el modelo ganador, la clase con mayor F1 fue sunflowers (0.9048) y la de menor F1 fue roses (0.8317). La brecha absoluta entre F1 macro de validación y prueba fue 0.0219; este valor, junto con las curvas de aprendizaje, permite valorar la generalización sin basarse solamente en Accuracy.

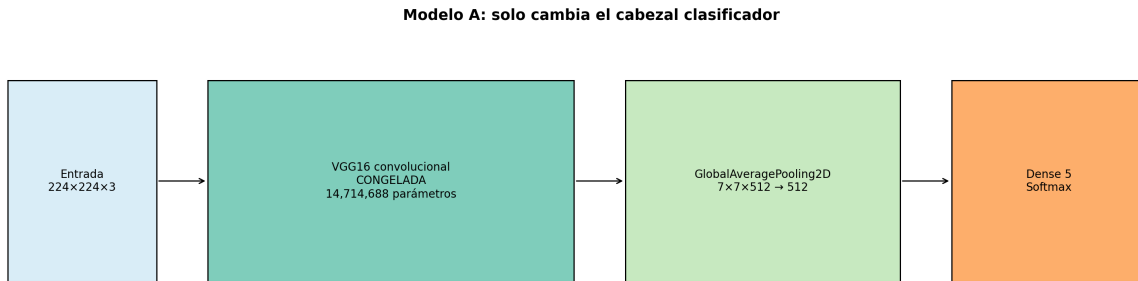


Figura 5: Diagrama exportado por el Train del Modelo A.

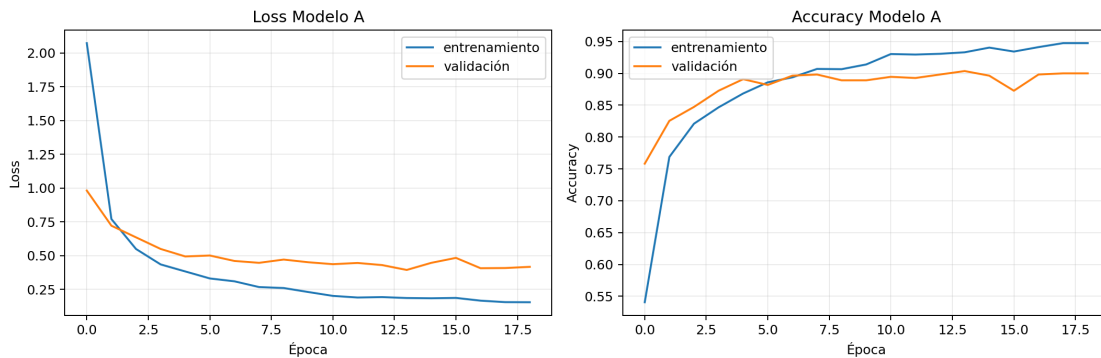


Figura 6: Curvas de entrenamiento y validación del Modelo A.

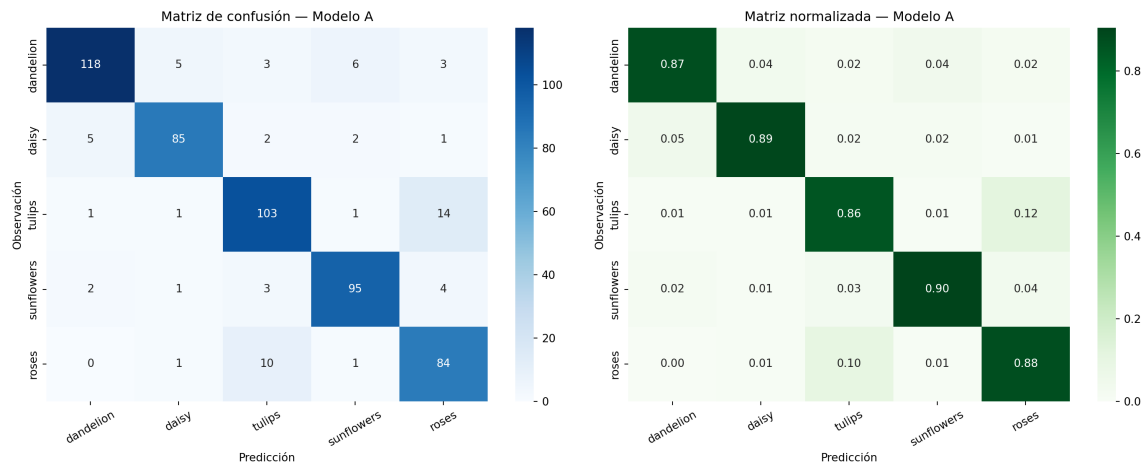


Figura 7: Matrices de confusión del Modelo A.

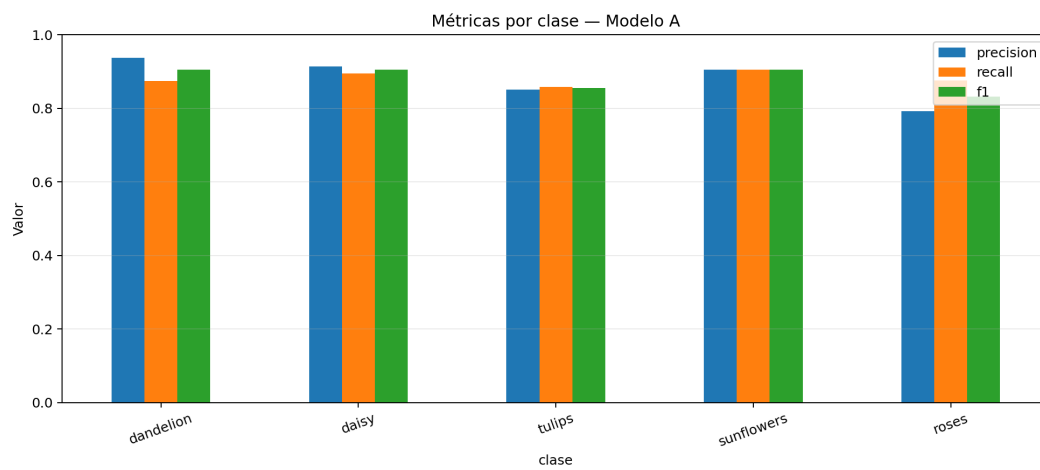


Figura 8: Precision, Recall y F1 por clase del Modelo A.

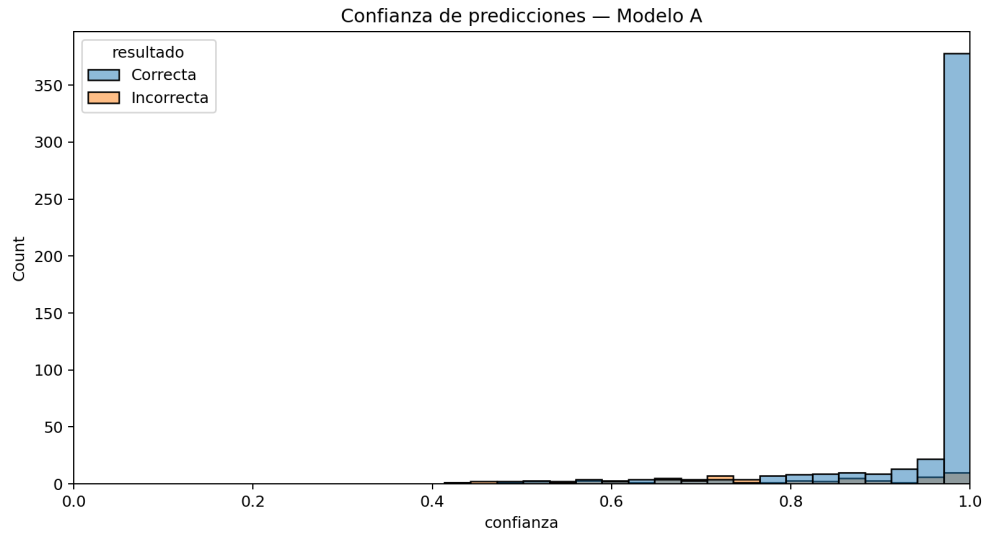


Figura 9: Histograma de confianza del Modelo A.

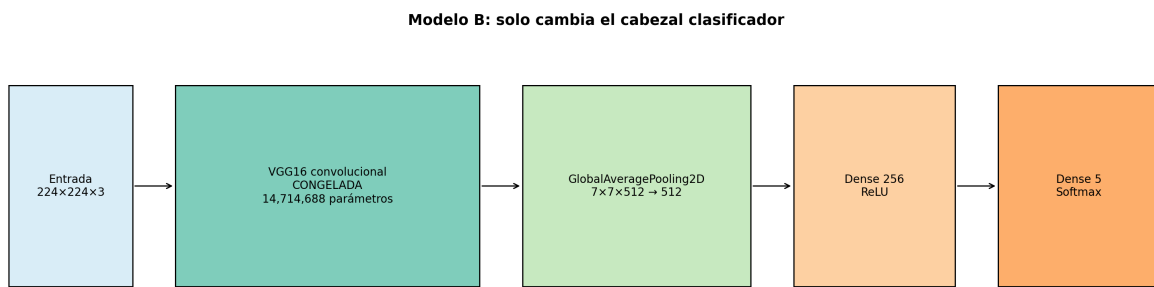


Figura 10: Diagrama exportado por el Train del Modelo B.

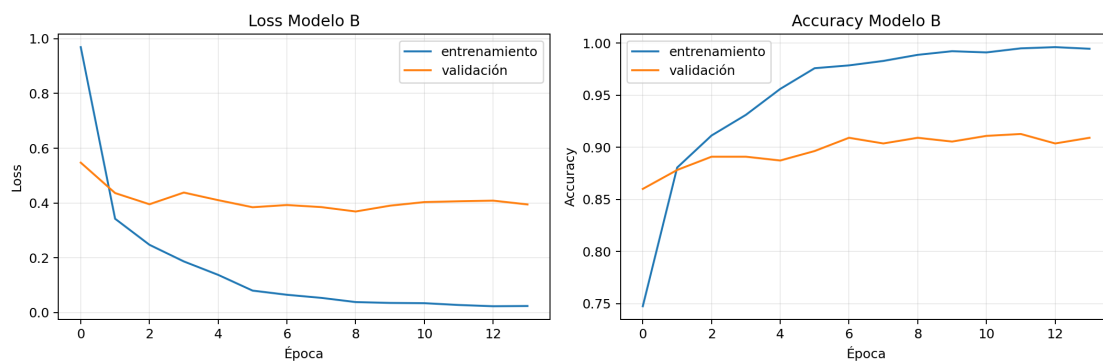


Figura 11: Curvas de entrenamiento y validación del Modelo B.

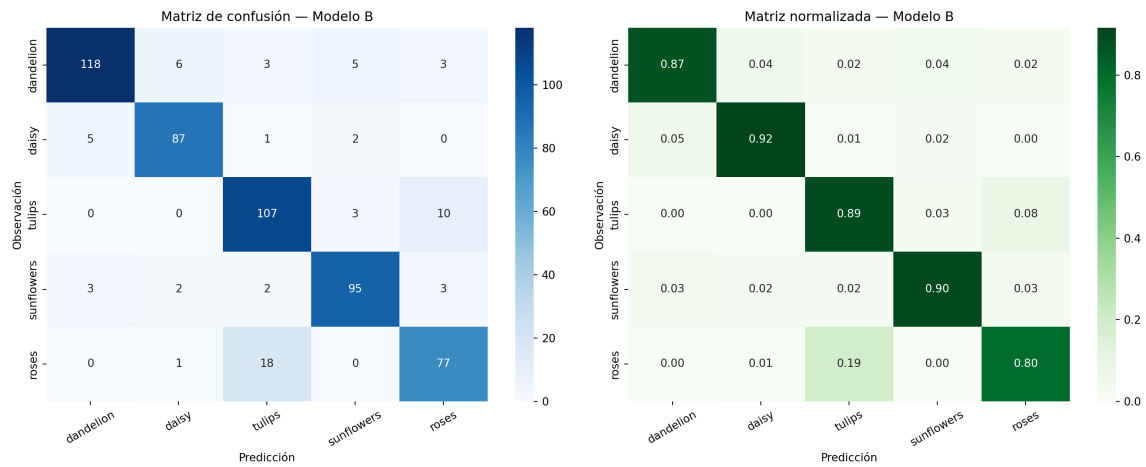


Figura 12: Matrices de confusión del Modelo B.

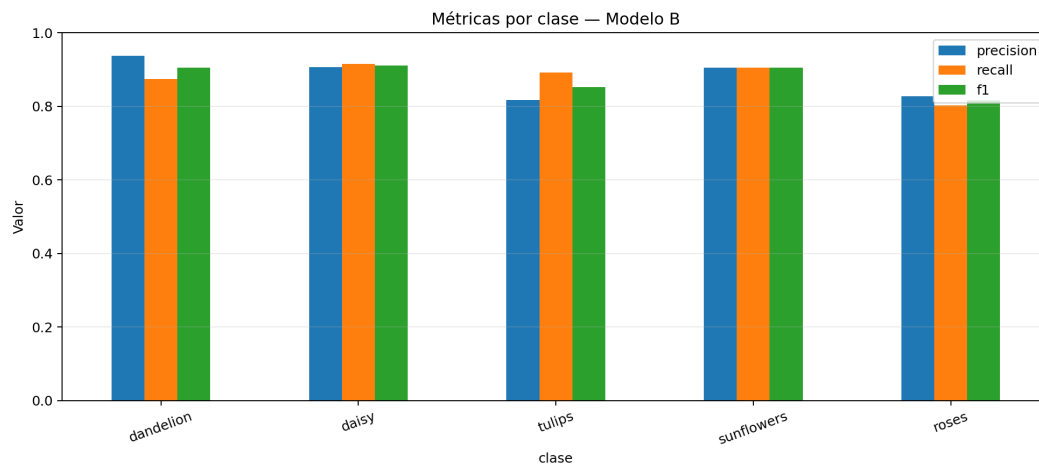


Figura 13: Precision, Recall y F1 por clase del Modelo B.

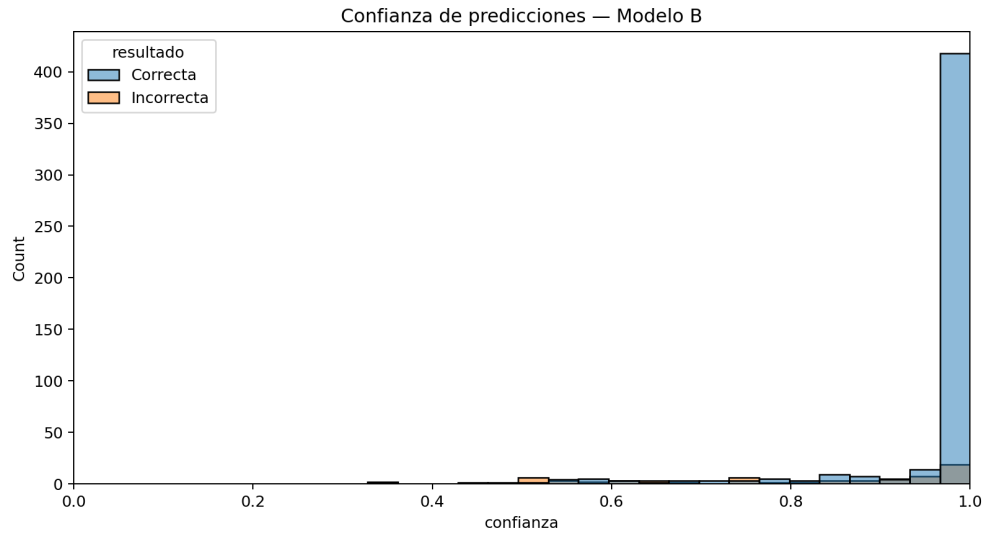


Figura 14: Histograma de confianza del Modelo B.

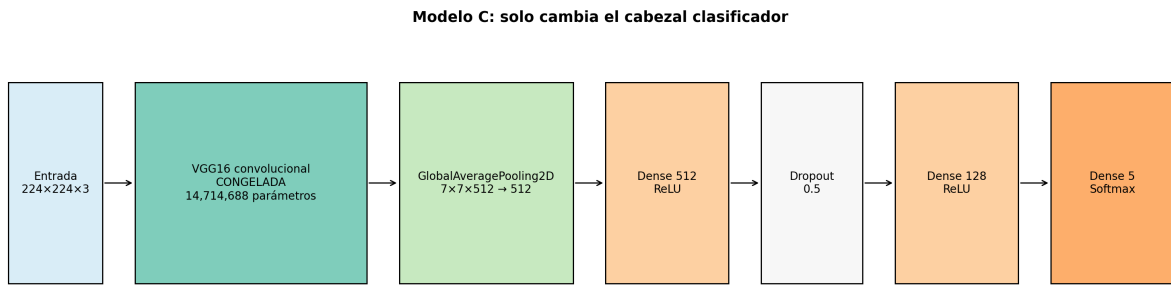


Figura 15: Diagrama exportado por el Train del Modelo C.

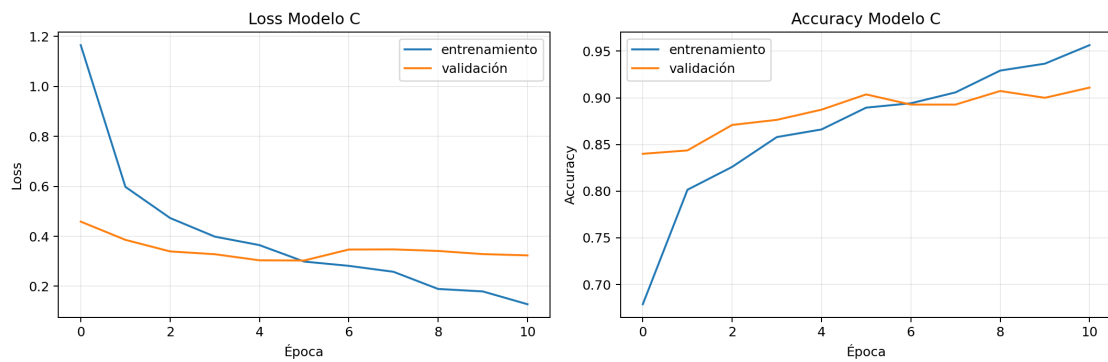


Figura 16: Curvas de entrenamiento y validación del Modelo C.

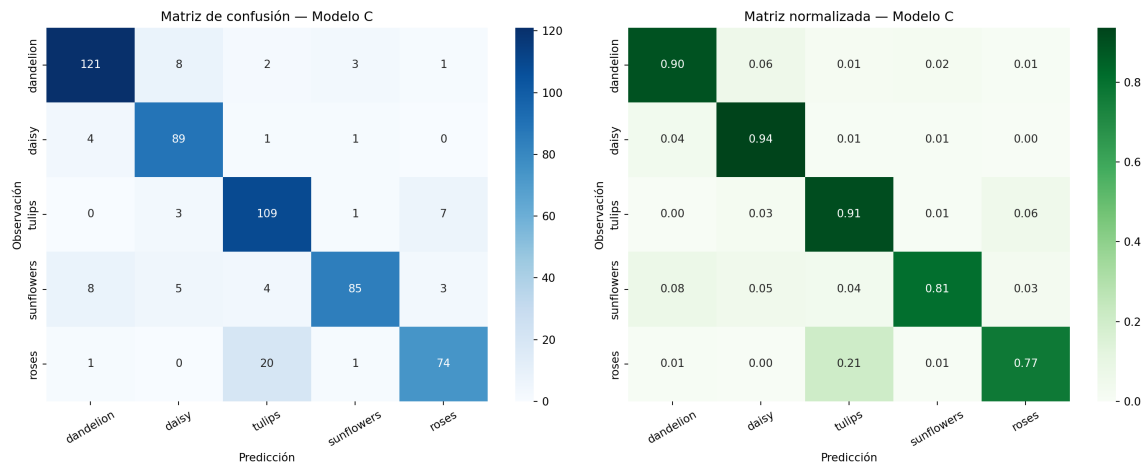


Figura 17: Matrices de confusión del Modelo C.

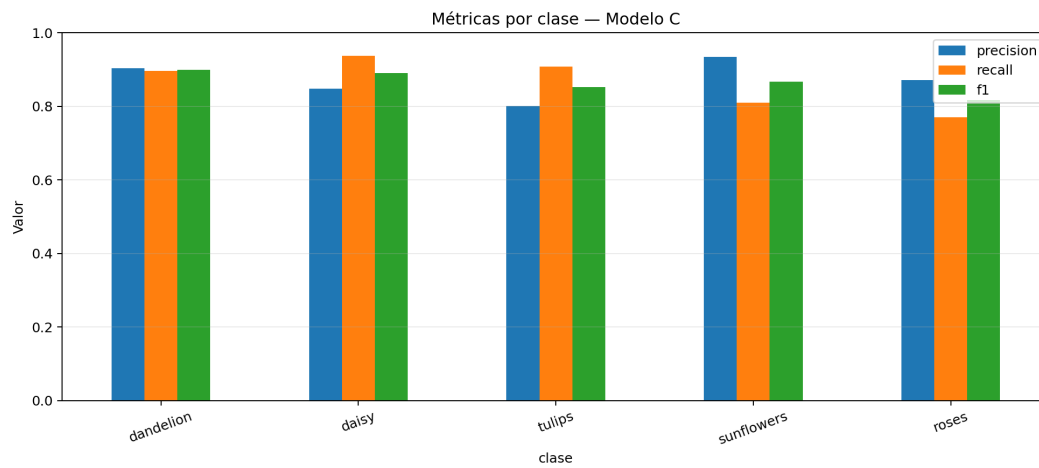


Figura 18: Precision, Recall y F1 por clase del Modelo C.

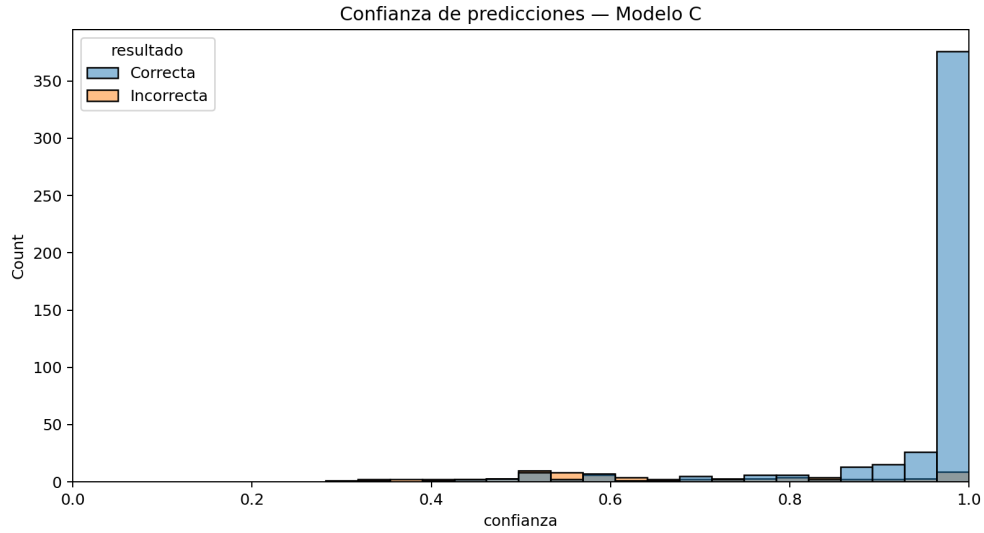


Figura 19: Histograma de confianza del Modelo C.

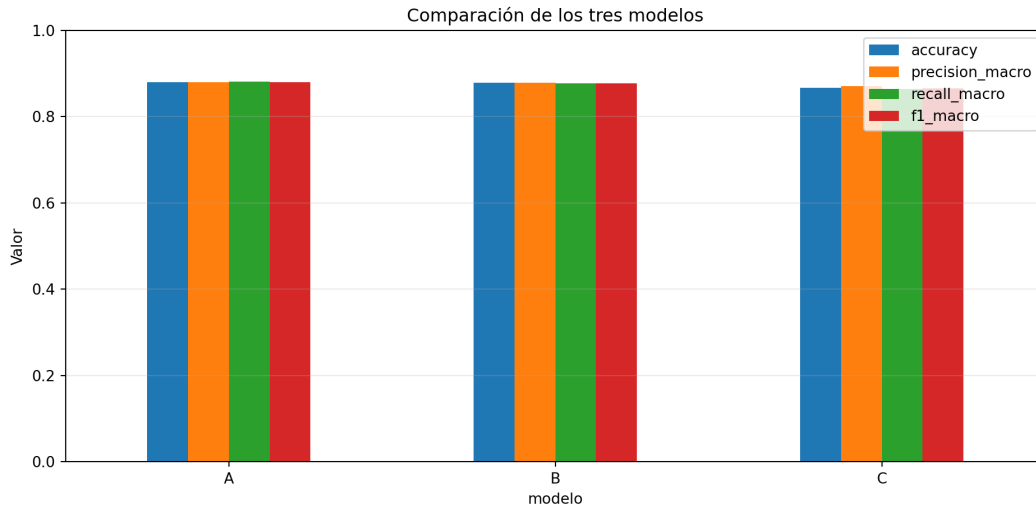


Figura 20: Comparación global de los tres modelos.

6. Discusión

Las curvas permiten contrastar convergencia y brecha entre entrenamiento y validación; las matrices muestran qué pares de flores se confunden. Los histogramas de confianza permiten detectar errores seguros, especialmente relevantes al comparar mayor capacidad contra regularización. La discusión final debe basarse en los archivos generados, no en valores estimados.

7. Conclusión

La selección sigue una regla fijada previamente: mayor F1 macro de prueba, seguido por Accuracy, menor brecha de generalización y menor complejidad. El Modelo A ofrece el mejor desempeño global. Alcanzó Accuracy de 0.8802, Precision macro de 0.8798, Recall macro de 0.8814 y F1 macro de 0.8799. Superó en F1 macro al Modelo B (0.8775) y al Modelo C (0.8652), y presentó la menor brecha validación–prueba (0.0219). Además, necesitó únicamente 2,565 parámetros entrenables, por lo que la mayor complejidad de los otros cabezales no produjo una mejora. Sus principales limitaciones aparecen en roses, con F1 de 0.8317; la matriz revela que la confusión dominante ocurre entre tulips y roses. En consecuencia, se elige el Modelo A por ofrecer la mejor combinación de desempeño, generalización y simplicidad.

8. Reproducibilidad

Se entregan seis notebooks Colab (Train y Test por modelo), configuración, manifiesto de splits, historiales, predicciones, métricas, figuras, modelos `.keras`, mejores pesos y hashes SHA-256. Cada Test carga exclusivamente el modelo producido por su Train correspondiente.

Referencias

- [1] TensorFlow Team. *tf_flowers, TensorFlow Datasets Catalog*. https://www.tensorflow.org/datasets/catalog/tf_flowers.
- [2] K. Simonyan y A. Zisserman. *Very Deep Convolutional Networks for Large-Scale Image Recognition*, 2015.